IDS Feature Selection & Hyperparameter Optimization using various Metaheuristic Algorithms

Mahmoud Yasser Sallam
School of Computer & Mathematical Sciences
University of Nottingham, Malaysia
Selangor, Malaysia
efyms24@nottingham.edu.my

Chan Jun Xi
School of Computer & Mathematical Sciences
University of Nottingham, Malaysia
Selangor, Malaysia
hfyjc22@nottingham.edu.my

Abdelrahman Sameh Fahmy
School of Computer & Mathematical Sciences
University of Nottingham, Malaysia
Selangor, Malaysia
hfyae3@nottingham.edu.my

Omair Ahmad
School of Computer & Mathematical Sciences
University of Nottingham, Malaysia
Selangor, Malaysia
hfyoa3@nottingham.edu.my

***Abstract*—Network Intrusion Detection Systems (NIDS) serve as a critical component of any modern cybersecurity infrastructure, being implemented in tandem with firewalls, Intrusion Prevention Systems (IPS) and Security Information and Event Management (SIEM) Systems, however it heavily depends on various factors such as dataset selection and feature selection, with serious repercussions if the IDS/NIDS is misconfigured. This paper investigates the application of five metaheuristic optimization algorithms ranging from Genetic Algorithm (GA) to a combined optimization algorithm Lion Optimization Algorithm-Virus Colony Search (LOA-VCS), this is done for feature selection and hyperparameter tuning of Random Forest and XGBoost classifiers on the CICIDS2017 dataset. Experimental results demonstrate that all metaheuristic enhanced models achieve overwhelmingly positive results, exceeding 99.7% accuracy while reducing feature dimensionality by 49-62%. This additionally discusses security implications, exploration-exploitation trade-offs and deployment considerations for ML-based intrusion detection.**

Keywords—Intrusion Detection Systems (IDS), Cybersecurity, CICIDS2017, Metaheuristic Algorithms, Genetic Algorithm (GA), Particle Swarm Optimization (PSO), Lion Optimization Algorithm (LOA), Virus Colony Search (VCS), Random Forest, XGBoost, Feature Selection, Firewalls, Intrusion Prevention System (IPS)

I. Introduction

Intrusion Detection Systems (IDSs), particularly Signature-based & Anomaly-based Network Intrusion Detection Systems (NIDS), remain an important aspect in network security as they work alongside other threat detection and response solutions, such as Security Information and Event Management [SIEMs] Systems and Intrusion Prevention Systems [IPSs] by detecting known malicious or suspicious activities [12][13][14]. Note that an IDS is implemented in tandem with firewalls and IPSs as they are primarily responsible for identification and classification [12].

The continuous research efforts concentrated on implementing machine learning and metaheuristic optimization algorithms on NIDSs demonstrated promising results in optimizing feature selection procedures and hyperparameters in IDSs and concerning trade-offs when incorporating overtly complicated solutions which may not prove well in regard to generalizability [3].

A. Dataset Details

The CICIDS2017 dataset [1] was used as the benchmark for all experiments. This dataset was released by the Canadian Institute for Cybersecurity (CIC) in late 2017 and contains network traffic captures with labelled benign and attack flows across seven attack categories (DoS, DDoS, Brute Force, Web Attacks, Infiltration, Botnet, PortScan, and Heartbleed). The pre-extracted CSV files (tabular flow features generated by CICFlowMeter) were used rather than raw Packet Capture files as the CSV have proven to be much easier to work with. All labels were binary encoded for this study: BENIGN was mapped to 0 (Normal), and all attack types were mapped to 1 (Attack).

The dataset was sourced directly from: https://www.unb.ca/cic/datasets/ids-2017.html, however due to the difficulties in consistently accessing it a copy of the set of folders that was used (MachineLearningCSV) alongside its MD5 checksum were saved and are available at: https://drive.google.com/drive/folders/1GywOJUIOoJxItGvTrpWCKPgzorHLp1L7?usp=drive_link.

B. Project Objectives

This paper presents the literature review, methodology, and implementation of a NIDS model enhanced by metaheuristic optimization algorithms, which allow them to dynamically adjust and adapt the hyperparameters and features selected while discussing the security and metaheuristic trade-offs, particularly between exploration and exploitation.

The project deliverables include a NIDS model with the following system architecture:

- Data Preprocessing Module: This module manages the CICIDS2017 CSV files and employs various preprocessing techniques, including but not limited to duplicates and invalid value removal, stratified dataset splitting, and Min-Max normalization.

- Base Model Module: This module contains the two main base model classifiers (Random Forest and XGBoost) that will be used to train and score the NIDS model against the metaheuristic algorithms.

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

- Metaheuristics Module: This module contains all five metaheuristic optimization algorithms with a shared abstract base class that provides common functionality between them.

- Integration Module: This module contains the Command Line Interface, which allows for the models to be trained and processed either with or without the metaheuristic algorithms.

- Evaluation & Visualization Module: This module contains all the performance evaluations, diagrams, graphs, and comparison plots to measure each model and its respective metaheuristic.

II. Background Research & Literature Review

Upon the release of the coursework, a guided literature review was conducted to understand the current state of research in intrusion detection systems, feature selection, and hyperparameter optimization to determine the most appropriate approach for this project in terms of scope and constraints, notably in choosing the metaheuristics, base models, and risk trade-offs.

The review commenced with examining the topics and documentation provided by IBM on cybersecurity and IDSs, which provided a foundational understanding of IDS architectures and operational principles. The review informed decisions regarding the specific scope of IDSs used and further goals [12][13][14].

The CICIDS2017 research paper provided an instrumental grasp on the dataset's goals, including features and extraction method, which would later prove to be essential in establishing baseline knowledge for developing a solution. The CICIDS2017 paper additionally employed Scikit-Learn's RandomForestRegressor for feature importance ranking, a methodology that informed the selection of Random Forest as one of the base classifiers in the present work due to its demonstrated high accuracy and low computational cost [1].

Further research led to findings regarding the security implications and risks, the BCCC-CSE-CIC-IDS2018/NTLFlowLyzer research paper revealed that the CICFlowMeter Analyzer was TCP-specific [2], making UDP-based attacks inappropriate for it and while this paper offered a new dataset with a potentially improved feature set and network analyser, the decision to remain with the CICIDS2017 dataset was due to the extensive research done on it and to ease the process of comparing results with other projects, furthermore it shed light on the issues of ineffective or aging Intrusion Detection Systems [1][2].

The implemented metaheuristics choices were inspired by [3], [4] & [5], where the LOA-VCS Metaheuristic Approach paper provided the initial framework to implement the LOA, VCS & Hybrid LOA-VCS metaheuristics, where for the hybrid framework specifically, the first half of the epochs employed LOA to explore the search space while the second half employed VCS to exploit the best candidates discovered, furthermore the contained preprocessing steps were of much use and were replicated in this project.

The modular implementation and architecture were inspired by the methodology inspired by [4], which demonstrated a modular approach towards their implementation that allowed for ease of scalability and was therefore adopted in the current work, facilitating swift iterative development and parameter adjustment through various configurations. The implementation and inclusion of the Particle Swarm Optimization Metaheuristic Algorithm, Cross-Validation, and Early Stopping/Convergence also came from the same body of literature, and while the implementation provided by [6] related to their usage of Synthetic Minority Over-sampling Technique was interesting, it was ultimately decided against since the class weights were adjusted to compensate for the class imbalance.

III. Methodology

A. Performance Metrics & Justification

The following performance Metrics were employed to evaluate the performance of the metaheuristic algorithms against the baseline models:

Precision: Quantifies how many recorded intrusions were genuine attacks and measures the amount of false positive attacks.

$$Precision = \frac{True\ Positive}{True\ Positive + Fals\quad Positive} \tag{1}$$

Recall: (Also known as Detection Rate) determines the ability to detect actual attacks and measures the number of false negative detections.

$$Recall = \frac{True\ Positive}{True\ Positive + False\ Negative} \tag{2}$$

Weighted & Macro F1-Scores: Provides a harmonic balance between precision and recall.

$$F1\text{-}Score = 2 \times \frac{Precision \times Recall}{Precision + Recall} \tag{3}$$

$$Weighted\ F1 = \sum_{I=1}^{N} w_i \times F1\ Score_i \tag{4}$$

$$Macro\ F1 = \frac{\sum_{I=1}^{N} F1\ Score_i}{n} \tag{5}$$

Accuracy: Measures the overall correctness of the model.

$$Accuracy = \frac{True\ Positive + T\qquad Negative}{TP + FP + TN + FN} \tag{6}$$

False Positive Rate: Measures the proportion of normal traffic incorrectly classified as attacks.

$$FPR = \frac{False\ Positive}{False\ Positives + True\ Negatives} \tag{7}$$

Computational Time: Measures the elapsed time per experimental run, encompassing both the metaheuristic search phase and the final model training phase.

The first three metrics (Precision, Recall & F1-Scores) are notably important, as they allow us to evaluate robustness to class imbalance in the dataset, while the remaining serve as

supplementary metrics to provide a more comprehensive evaluation of the project.

B. Preprocessing Pipeline

The following preprocessing techniques were employed sequentially to the raw CICIDS2017 CSV files:

- **Duplicate removal, Infinite handling, and missing value removal**: All eight CSV files were loaded and concatenated into a single DataFrame of approximately 2.5 million rows across approximately eighty features. Rows containing NaN values, infinities, and exact duplicates were removed.

  Binary label encoding for categorical variables: The variables were mapped to a binary classification where benign multi-classes were labelled as zero (0) and attacks were labelled as one (1).

- **Feature filtering on the training set. (low variance and high correlation thresholds):** Features that were found to be mostly constant (Low variance with a std < 0.01) in addition to variables that had a high correlation with each other ($|r| > 0.95$) were removed as they were deemed to be redundant.

  Stratified splitting: The cleaned data was separated into a 70/15/15 split between training, validation, and testing using stratified sampling to preserve the class distribution.

- **Min-Max Normalisation of numerical features:** All numerical features were scaled between [0...1] using a MinMaxScaler, which fit exclusively on the training set to prevent data leakage.

- **Class weighting:** Inverse frequency class weights were computed from the training set, initially the thought to implement SMOTE [Synthetic Minority Over-sampling Technique] in tandem with class weight modification was brought up but decided against as combining them would artificially induce overfitting and exaggerate the minority attack labels while utilizing class weightings would penalize incorrect classification of the minority labels (Attack instances) harshly which better suited the current goals.

C. Base Models

Two ensemble-based classifiers were selected as the base models for all experimental evaluations: Random Forest [RF] and XGBoost. Both models were employed in two capacities: as standalone baseline classifiers trained with all pre-processed features and default hyperparameters, and second, as the underlying classifiers whose feature subsets and hyperparameters were optimized by each metaheuristic optimization algorithm. The base model itself remained consistent throughout, while the metaheuristic algorithms functioned as optimizers of the model's input and configuration.

Random Forest is a bagging-based ensemble ML algorithm that combines multiple decision tree classifiers on random sub-samples of the dataset to improve the predictive accuracy and reduce over-fitting where possible [7]. The implementation used in this project is SciKit's RandomForestClassifier acquired from Scikit-Learn via: "from sklearn.ensemble import RandomForestClassifier" [8].

XGBoost (eXtreme Gradient Boosting) is a gradient boosted decision tree ML algorithm that initializes several "weak" learners (typically shallow decision trees) in a sequential manner where each successive tree is trained to minimize the residual errors of the cumulative ensemble by fitting to the negative gradient descent in function space [9] [10]. This is an optimization algorithm used to reduce error rates in ML models by adjusting the parameters where errors decrease the most, and it combines multiple weaker models to create a performant model. The implementation utilized in this project is from the XGBoost Python library, using from xgboost import XGBClassifier [9][10][11].

A note is that XGBoost is implemented on top of a core C++ Library [11], while RFClassifier is built using a hybrid between Python, Cython/C++; this architectural difference contributes to XGBoost's substantially lower training times, where its parallelized tree-building and efficient memory access pattern over large datasets offer significant computational advantages [11].

Both models were evaluated using 3-fold stratified cross-validation on the training set during the metaheuristic optimization phase, where a 5% stratified subsample of the training data was used for fitness evaluation during the metaheuristic search to prevent ArrayMemoryErrors with NumPy and "Out Of Memory" issues that have taken place when running with various other parameters, including a higher stratified subsample percentage. This strategy is justified by the notion that metaheuristic optimizations require only correct fitness rankings rather than absolute fitness values. This is combined with the fact that the final model training and evaluation were always conducted on the full training and test sets to confirm the generalizability of the optimized hyperparameters and features selected.

D. Metaheuristic Optimization Algorithms

All the optimization algorithms implemented here share a common solution encoding scheme where each candidate solution is represented as a continuous vector in the range [0...1], and the remaining dimensions are linearly mapped to the model-specific hyperparameter bounds. This unified encoding is implemented in a shared abstract class (MetaheuristicBase), which provides common functionality for solution decoding, fitness evaluation through weighted F1-scores that are cross-validated 3-fold, and evaluation caching to mitigate redundant computations.

Five metaheuristic algorithms were implemented to optimize feature selection and hyperparameter tuning for the base classifiers; their details are described below:

1. Genetic Algorithm (GA): A population-based evolutionary approach inspired by natural selection and genetic inheritance. The algorithm maintains a population of candidate solutions (entities/individuals) that evolve across successive generations through the application of selection, crossover, and mutation operators.

2. Particle Swarm Optimization (PSO): A population-based, stochastic metaheuristic that simulates the collective behaviour of a group of particles navigating

a search space where each particle maintains its respective candidate solution, velocity vector, and a memory of its personal best position (pbest). PSO is known for its fast convergence and simplicity, but it suffers from premature convergence in high-dimensional problems. The swarm collectively tracks the global best position (gbest) discovered by any particle. At each iteration, the velocity of each particle is updated according to three components: an inertia term (w), a cognitive component (c_1), a social component (c_2), and random numbers (r_1 & r_2) using the following equation:

$$v = wv + c_1 r_1 (pbest - x) + c_2 r_2 (gbest - x) \quad (8)$$

3. Lion Optimization Algorithm (LOA): A population-based, nature-inspired metaheuristic that models the social behaviour of lions, distinguishing between pride members that exhibit cooperative, exploitation-focused behaviour and nomadic individuals that explore the search space independently [3].

4. Virus Colony Search (VCS): A population-based, nature-inspired metaheuristic that simulates the biological processes behind virus infection, diffusion, and immune response. The algorithm operates in two alternating phases [3]. The diffusion-infection phase represents two viable options; each virus moves towards the best-known solution (gbest) via a random vector or diffusion, which runs it via a Gaussian random walk. The viral diffusion can be represented via the following equation (9) & code snippet in Fig. 1.:

```
step = np.random.rand(self.solution_dim) * (gbest_position - positions[i])

New_position[i] = positions[i] + step
```

Fig. 1. VCS Code Snippet

$$x_i^{t+1} = x_i^t + \delta.rand() \times (gbest - x_i^t) \quad (9)$$

The subsequent immune response phase simulates the host organism's immune system by locating and replacing the weakest 20% of the population with newly generated random solutions, which are then re-evaluated while preserving the best solutions through elitism.

5. LOA-VCS Hybrid: Finally, the LOA-VCS Hybrid is a mixture between both of them proposed by [3], where they employed the first half of their runs to LOA to broadly explore the solution space, while the second half was allocated to VCS to refine and exploit the best candidates discovered [3].

E. Integration Module

The Integration Module consists of a Command-Line Interface that connects the pre-processed data from the preprocessing pipeline to the base models and metaheuristic algorithms, where upon initiation the user is prompted to select their desired base model and subsequently select a metaheuristic algorithm including the option to run without utilizing any metaheuristic algorithms as a baseline run and a "Run All" option that sequentially executes all the metaheuristics without requiring further input.

An abstract parent class was created, which ensured that all metaheuristics shared specific parameters such as their population size (20), maximum generations (15), and subsample ratio (5%), where the fitness function consisted of a weighted F1-score via 3-fold stratified cross-validation.

Upon execution of the optimizer, it would work on iteratively refining the population based on the fitness results, where once the optimal feature mask and hyperparameter configuration set are discovered (referred to as the best solution), they would be saved upon completion for further use.

The identified best solution would then be used to train a fresh model instance on the complete training dataset to ensure that the model generalizes well and provides a realistic metric of generalization capabilities, after which the results would be saved to a time-stamped run directory.

F. Evaluation & Visualization Module

The Evaluation Module operates as an analysis and visualization tool that aggregates results from all completed experiments in the "results/" folder and generates a summary of their respective performance metrics, as discussed earlier, along with per-run confusion matrices that evaluate the predicted and true classes and comparative bar plots.

IV. Discussion and Results

A. Algorithm Justification

The selection of the five metaheuristic algorithms was guided by three criteria's: established effectiveness in classification tasks, diversity of search strategies, and relevance to the IDS feature selection domain as informed by the literature review.

The Genetic Algorithm (GA) was included as a well-established evolutionary baseline, especially with it being taught as one of the first metaheuristics in the AI Methods module, tournament selection at (k=3) was employed alongside; two-point crossover (rate 0.8), and random-reset mutation (rate 1/n), with elitism as to preserve the top two individuals per generation. GA's proven effectiveness in combinatorial feature selection problems [3][4] and its intuitive parallelism made it a natural first choice for benchmarking against more recent approaches.

Particle Swarm Optimization (PSO) was chosen as the second metaheuristic for its swarm-intelligence paradigm, which differs from GA's evolutionary operators. The implementation employs linearly decaying inertia weights (0.9 to 0.4), cognitive and social coefficients (c1=c2=2.0), and velocity clamping at ±0.5. PSO's capability at leading continuous-valued search aligns naturally with the joint feature-hyperparameter encoding used in this work, as demonstrated by [4] in comparable IDS optimization tasks.

The Lion Optimization Algorithm (LOA) was selected directly from the framework proposed in [5], structured around an 80/20 pride-to-nomad ratio. Pride lions perform exploitation by moving towards the alpha (global best) with Gaussian noise that decayed over generations, while nomad lions maintain exploration through random re-initialization. This social hierarchy naturally balances the exploration-exploitation trade-off without requiring explicit scheduling.

Virus Colony Search (VCS) provided a complementary biological metaphor thanks to its three-phase cycle of

Gaussian diffusion (exploration) and infection of best hosts (exploitation) and additionally its immune replacement of the weakest 20% of the population. The decreasing diffusion variance provides a natural annealing schedule that progressively refines candidate solutions [5].

The LOA-VCS hybrid is the most novel contribution as it was directly inspired by [5]. The first half of its generations employ LOA's pride/nomad dynamics for broad exploration, while the second half transitions to employing VCS's infection-diffusion-immune cycle for fine-grained exploitation. This sequential hybridization was hypothesized to combine LOA's strong global search with VCS's local refinement capabilities, and the experimental results confirmed that it achieved competitive performance, particularly with Random Forest, where it achieved the second highest accuracy of 99.80% while reducing features from 45 to 22.

B. Security Risks, Mitigations & Trade-Offs

The deployment of an ML-based NIDS introduces several security considerations that go way beyond classification accuracy and performance metrics. These risks are discussed below:

Threat Detection Tools: First and foremost, it is important to acknowledge that IDSs are not intended to be used solely by themselves, but as a part of a network of solutions implemented to mitigate and prevent threats and attacks [12][14], according to IBM, IDSs are typically expected to be integrated with robust firewalls, IPSs, and SIEMs to assist SOC analysts in detecting and responding to threats [12][13][14].

Dataset Age: While the dataset was comprehensive at the time of its release (2017), it does not account for attack methodologies that have emerged recently, especially with AI-driven attacks arising from advanced persistent threats (APTs) utilizing generative AI to exploit and create zero-day exploit attacks, this is supported by IBM noting that adversaries are increasingly leveraging AI to automate attacks and evade detection at scale [13], this is a development that datasets from 2017 will prove hard to capture even given its Anomaly-based detection methodology and may exhibit lowered detection performance against newer attack vectors.

Encryption and Evasion Limitations: While discussing the details of this project with IT GRC/SOC Manager & CISO Yasser Sallam, it was noted that IDSs operating on network flow metadata cannot examine encrypted payloads, and traffic encapsulated via HTTPS/TLS may contain malicious content that flow-level features alone cannot detect, as the IDS lacks access to the decryption keys. Further IDS evasion tactics, which were discussed, included traffic fragmentation to obscure malware signatures from Signature-based & Hybrid implementations and payload encryption.

Overfitting Risk from Metaheuristic Search: The metaheuristic optimization process, which evaluates candidate solutions using 3-fold cross-validation on a 5% stratified subsample of the training data, introduces a major risk of overfitting to the specific characteristics of this subsample. Therefore, three safeguards were implemented to mitigate it, which consisted of: stratified subsampling that ensures the class distribution of the subsample matches the full dataset, cross-validation provides a degree of regularization against sample overfitting, and finally, the model is always retrained on the complete training set and evaluated on a held-out test set that was never exposed to the optimization process.

Feature Reduction & Information Loss: The metaheuristic optimization process reduced the feature set significantly in some cases, while this reduction yields computational efficiency benefits, it adversely discards information that might prove to be relevant in detection-specific attack subtypes, particularly given the NIDS anomaly-based detection role. And as binary classification was opted for this project, these losses may be masked and therefore employing multi-class evaluation in future attempts would provide richer insight into the impact of feature reduction on performance & evaluation metrics.

C. Metaheuristic Effectiveness & Performance Comparison

All the experimental runs were conducted on both Google Colab and a local computer equipped with an Intel I5-14400F CPU, an NVIDIA RTX 4070 GPU, 32GB DDR4 RAM, and Windows 11 [64-bit]. The project framework was implemented in Python 3.14.3 using SciKit-Learn, XGBoost, NumPy, Pandas & more, the complete list of which is accessible in the GitHub repository.

Table I and II present the complete performance metrics for the Random Forest and XGBoost base models across all experimental configurations. A summary of the confusion matrix values extracted from the per-run confusion matrices (Figs. 9 & 10) are provided in the table III.

TABLE I. RANDOM FOREST PERFORMANCE COMPARISON

Metric	Metaheuristic Algorithm					
	Baseline	**GA**	**PSO**	**LOA**	**VCS**	**LOA-VCS**
Accuracy	0.9978	0.9979	0.9975	0.9982	0.9979	0.9980
Precision	0.9940	0.9903	0.9876	0.9943	0.9946	0.9943
Recall	0.9932	0.9972	0.9975	0.9950	0.9930	0.9941
F1 Binary	0.9936	0.9937	0.9926	0.9946	0.9938	0.9942
F1 Weighted	0.9978	0.9979	0.9975	0.9982	0.9979	0.9980
FPR	1.23e-3	1.98e-3	2.54e-3	1.16e-3	1.10e-3	1.16e-3
Features	45	23	26	22	22	22
Train Time (s)	124.55	78.68	92.55	305.38	256.39	301.19

TABLE II. XGBOOST PERFORMANCE COMPARISON

Metric	Metaheuristic Algorithm					
	Baseline	**GA**	**PSO**	**LOA**	**VCS**	**LOA-VCS**
Accuracy	0.9991	0.9991	0.9991	0.9991	0.9991	0.9991
Precision	0.9949	0.9948	0.9948	0.9949	0.9949	0.9949
Recall	0.9997	0.9997	0.9997	0.9996	0.9996	0.9996
F1 Binary	0.9973	0.9973	0.9973	0.9972	0.9972	0.9972
F1 Weighted	0.9991	0.9991	0.9991	0.9991	0.9991	0.9991
FPR	1.04e-3	1.05e-3	1.06e-3	1.05e-3	1.04e-3	1.05e-3
Features	45	20	22	17	25	17
Train Time (s)	13.35	19.69	8.40	15.29	15.34	13.25

Both base models achieved strong classification performance without metaheuristic optimization when trained on the full set of 45 preprocessed features. The Random Forest baseline achieved an accuracy of 99.78%, a weighted F1-score

of 0.9978, and a false positive rate of 0.00123, while XGBoost demonstrated stronger baseline performance across all metrics, achieving an accuracy of 99.91%, a weighted F1-score of 0.9991, and a notably lower FPR of 0.00104.

The metaheuristic algorithms produced measurable but modest improvements for the Random Forest classifier, where LOA emerged as the best-performing optimiser, achieving the highest accuracy (99.82%), binary F1-score (0.9946), and weighted F1-score (0.9982) while reducing the feature set from 45 to 22 features. This represents an improvement of 0.04 percentage points in accuracy and a reduction in false negatives from 436 to 320, confirming that the pride-nomad social hierarchy was effective at identifying a compact, high-quality feature-hyperparameter configuration.

GA and PSO exhibited a distinct trade-off pattern, where both algorithms substantially improved recall (GA: 0.9972, PSO: 0.9975) relative to the baseline (0.9932), reducing false negatives to 181 and 158 respectively. However, this improvement came at the cost of increased false positives, with GA's FPR rising to 0.00198 and PSO's to 0.00254, the highest of any configuration. This behaviour suggests that GA and PSO drove the model towards more aggressive classification boundaries that favour detection sensitivity over false alarm avoidance, a trade-off that may be acceptable in high-security environments where missing an attack carries a greater cost than investigating a false alert, however in operational SOC contexts where alert fatigue is a documented concern [13][14], the elevated FPR would be undesirable.

VCS achieved the lowest FPR of any RF configuration at 0.00110, improving upon the baseline's 0.00123, but recorded the highest false negative count among the metaheuristics (446, slightly above the baseline's 436), this suggests that VCS's immune response mechanism, which replaces the weakest 20% of the population, favoured solutions that prioritised specificity over sensitivity.

LOA-VCS produced results that fell between its constituent algorithms across most metrics, achieving an F1-score of 0.9942, an FPR of 0.00116, and 22 selected features, which is consistent with the expected behaviour of the phased exploration-exploitation design proposed in [5], where LOA's broad search in the first half of generations identifies promising regions that VCS subsequently refines.

An important observation is that all metaheuristic-optimised RF models achieved comparable or improved performance while using approximately half the original features (22 to 26 features from 45), furthermore GA achieved the fastest final training time at 78.68 seconds, a 36.8% reduction from the baseline's 124.55 seconds, demonstrating the practical computational benefit of dimensionality reduction for Random Forest.

In contrast to Random Forest, the metaheuristic algorithms produced negligible performance differences for XGBoost. As shown in Table III, accuracy remained at 99.91% across all configurations, weighted F1-score held at 0.9991, and FPR varied by only 0.00002 between the best and worst cases. The confusion matrices confirm this stability, where false negative counts ranged from 16 (PSO) to 28 (LOA-VCS), a variation of only 12 misclassifications out of approximately 378,000 test samples. This stability indicates that XGBoost's internal regularisation mechanisms, including its gradient-based sequential learning, L1/L2 regularisation, and built-in feature importance weighting, already approximate an optimal feature-hyperparameter configuration for this dataset, and the metaheuristic search operating on a 5% subsample with 3-fold cross-validation was unable to discover configurations that meaningfully improved upon XGBoost's defaults when evaluated on the full test set.

However, the metaheuristics did achieve significant feature reduction for XGBoost without degrading performance, where LOA and LOA-VCS both reduced the feature set to 17 features (a 62.2% reduction from 45), while GA selected 20 features, all with no measurable loss in detection accuracy. This finding has practical implications for deployment scenarios, as a model operating on 17 features requires less data preprocessing, consumes less memory, and evaluates faster at inference time than one using 45 features.

TABLE III. CONFUSION MATRIX SUMMARY

Configuration	Classification Outcomes			
	TN	FP	FN	TP
RF Baseline	313,874	385	436	63,425
RF + GA	313,637	622	181	63,680
RF + PSO	313,462	797	158	63,703
RF + LOA	313,894	365	320	63,541
RF + VCS	313,912	347	446	63,415
RF + LOA-VCS	313,895	364	376	63,485
XGB Baseline	313,931	328	17	63,844
XGB + GA	313,928	331	19	63,842
XGB + PSO	313,927	332	16	63,845
XGB + LOA	313,929	330	27	63,834
XGB + VCS	313,931	328	26	63,835
XGB + LOA-VCS	313,929	330	28	63,833

A summary of the confusion matrix values extracted from the per-run confusion matrices (Figs. 9 & 10) are provided in Table III.

The confusion matrices further illustrate the gap between XGBoost's baseline and Random Forest's baseline, where XGBoost's baseline recorded only 17 false negatives compared to Random Forest's 436, indicating substantially stronger attack detection capability without any optimisation applied. This is consistent with XGBoost's architectural advantages discussed in the Base Models section, particularly its sequential error-correction through gradient boosting and its parallelised C++ core, which together produce a more refined decision boundary [9][10][11].

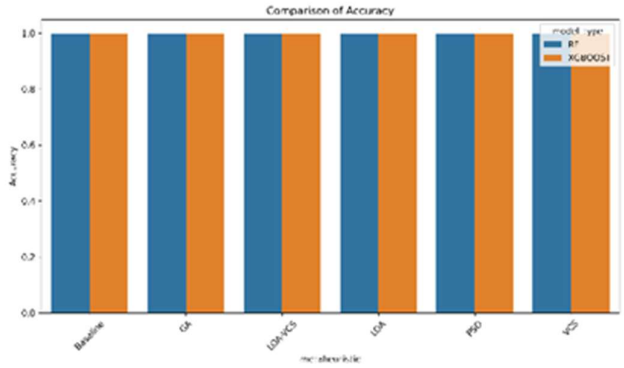

Fig. 2. Accuracy Comparison Chart

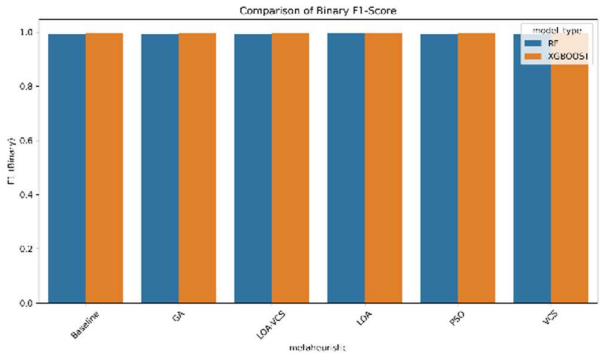

Fig. 3. F1 Score Comparison Chart

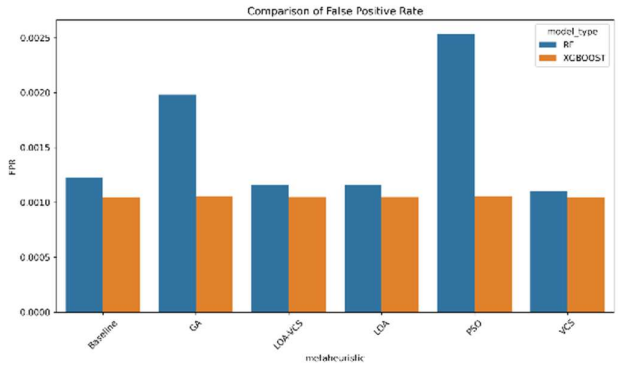

Fig. 4. False Positive Rate Comparison Chart

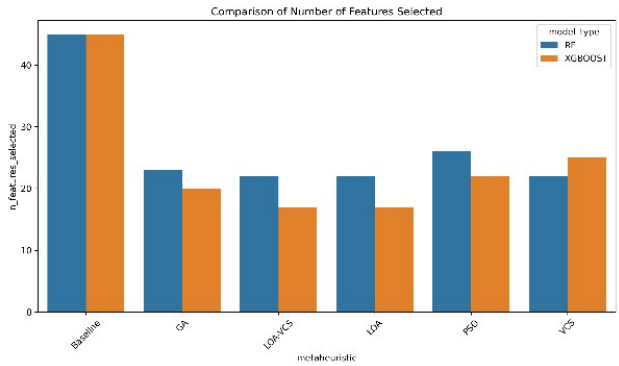

Fig. 5. No.# of Features Selected Comparison Chart

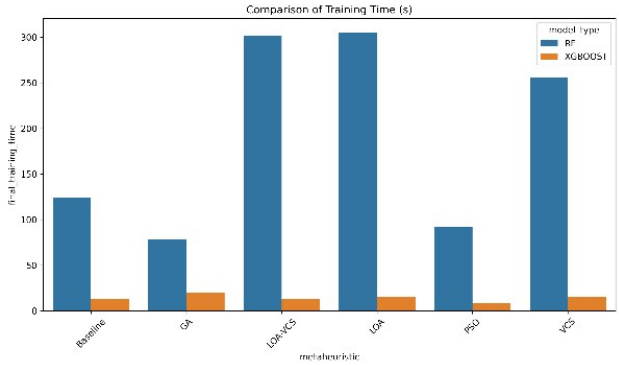

Fig. 6. Training Time Comparison Chart

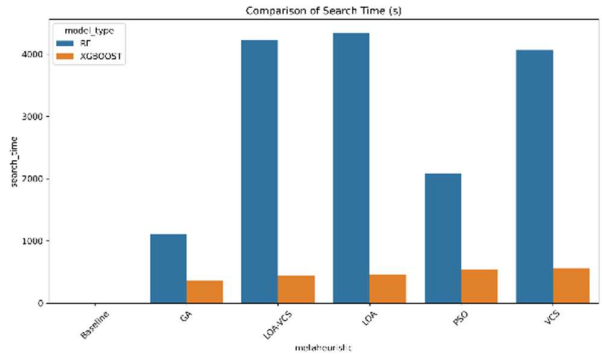

Fig. 7. Search Time Comparison Chart

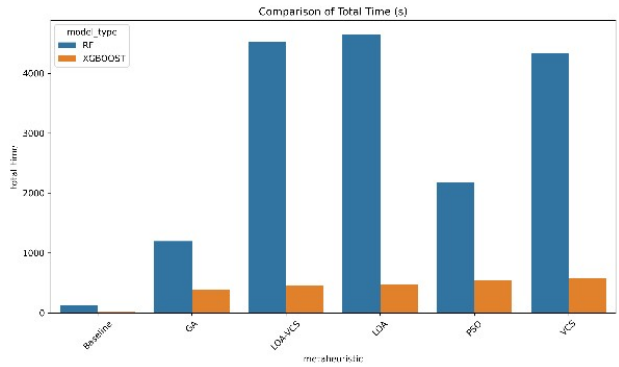

Fig. 8. Total Time Comparison Chart

Fig. 9. RF Confusion Matrices

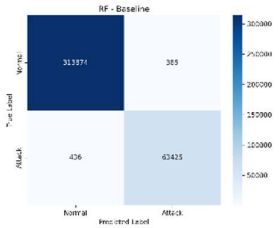

Fig. 9.1. Baseline

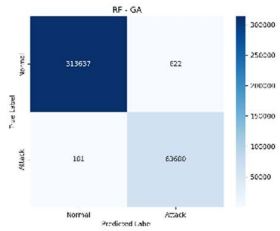

Fig. 9.2. GA

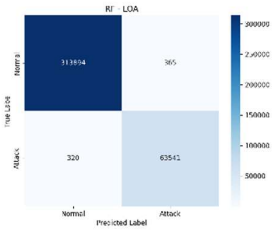

Fig. 9.3. LOA

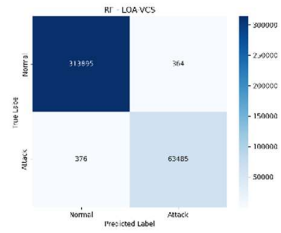

Fig. 9.4. LOA-VCS

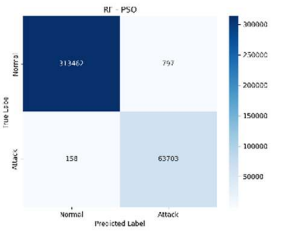

Fig. 9.5. PSO

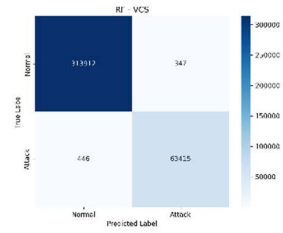

Fig. 9.6. VCS

Fig. 10. XGBoost Confusion Matrices

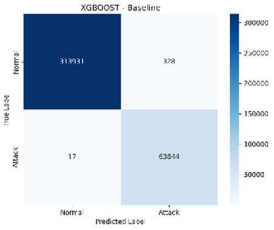

Fig. 10.1 Baseline

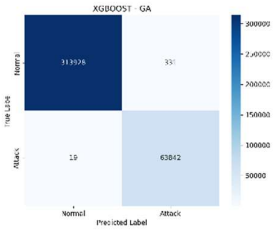

Fig. 10.2. GA

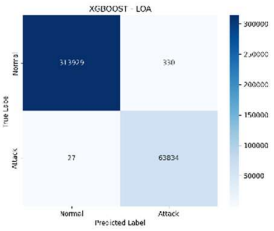

Fig. 10.3. LOA

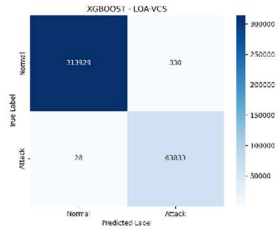

Fig. 10.4. LOA-VCS

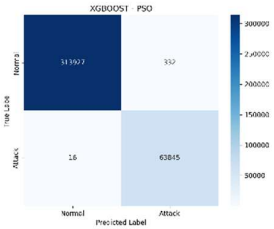

Fig. 10.5. PSO

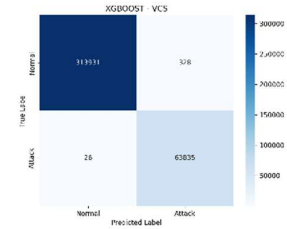

Fig.10.6. VCS

XGBoost consistently outperformed Random Forest across all configurations, where the performance gap was most pronounced in false negative rates: XGBoost's worst configuration (LOA-VCS, 28 FN) still outperformed RF's best configuration (LOA, 320 FN) by an order of magnitude. XGBoost also demonstrated a substantial computational advantage, with final training times ranging from 8.40 to 19.69 seconds compared to RF's 78.68 to 305.38 seconds, a difference attributable to XGBoost's parallelised C++ backend as discussed earlier [11]. The total experiment times (search plus final training) reveal a further distinction, where XGBoost's metaheuristic runs completed in 387 to 579 seconds while RF's ranged from 1,194 to 4,649 seconds, this is because each fitness evaluation during the metaheuristic search requires training and cross-validating a model on the subsample, and RF's slower per-evaluation cost compounds across the population size (20) multiplied by the generation count (15), totalling 300 evaluations per run.

All metaheuristic algorithms converged on substantially reduced feature subsets regardless of the base model, where the average feature count across all optimised RF configurations was 23.0 while XGBoost configurations averaged 20.2, both down from 45 in the baselines. This consistent reduction of approximately 49% to 62% suggests that the CICIDS2017 feature space, even after the preprocessing pipeline's variance and correlation filtering, still contains considerable redundancy that the metaheuristic search was able to identify and remove without impacting classification performance. This aligns with the broader IDS literature, which has consistently shown that feature selection reduces the curse of dimensionality and computational overhead without sacrificing detection quality on well-structured datasets [1][5].

V. Conclusion

This paper presented the design, implementation and evaluation of a metaheuristic-enhanced Network Intrusion system, where five metaheuristic algorithms (GA, PSO, LOA, VCS and LOA-VCS) were applied to optimize feature selection and hyperparameter tuning for Random Forest and XGBoost classifiers on the CICIDS2017 dataset.

Our experimental results demonstrated that all the configurations achieved classification performance exceeding 99.7% accuracy. XGBoost consistently outperformed Random Forest across all configurations, achieving 99.91% accuracy with only 17 false negatives in its baseline, a result that the metaheuristic algorithms were unable to meaningfully improve upon due to XGBoost's internal regularization mechanisms already approximating an optimal configuration for this dataset. The metaheuristics proved more impactful for Random Forest, where LOA achieved the highest accuracy of 99.82% while reducing features count from 45 to 22, and GA and PSO demonstrated an interesting precision-recall trade-off by reducing the false negatives at the cost of the elevated false positive rates.

A key finding was the consistent feature reduction of 49% to 62% without degrading performance, where LOA and LOA-VCS reduced XGBoost's features from 45 to 17 with no measurable loss in detection accuracy, which has practical implications for the deployment as fewer features translate directly to reduced preprocessing overhead, lower memory consumption and faster inference times in high-throughput network environments.

However, several limitations must be acknowledged. Notably, that the CICIDS2017 dataset does not account for modern AI-driven attack methodologies [13], the CICFlowMeter tool's TCP-specificity leaves UDP-based attacks inadequately represented [2], and the implemented binary classification scheme may mask performance differences at the individual attack-type level. Future work could address these limitations by evaluating on newer datasets such as BCCC-CIC-IDS2017 [2], employing multi-class classification for per-attack-type analysis, running multiple seeds to assess statistical significance and incorporating adversarial robustness testing against the IDS evasion tactics discussed in this paper.

References

[1] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, 'Toward generating a new intrusion detection dataset and intrusion traffic characterization', in Proceedings of the 4th International Conference on Information Systems Security and Privacy, 2018.

[2] M. Shafi, A. H. Lashkari, and A. H. Roudsari, 'Toward generating a large scale intrusion detection dataset and intruders behavioral profiling using network and transportation layers traffic flow analyzer (NTLFlowLyzer)', J. Netw. Syst. Manag., vol. 33, no. 2, Apr. 2025.

[3] College of Information Technology, Cyber Security Department, Amman Arab University, Amman, Jordan, M. O. Nassar, and F. F. AL-Mashagba, 'Novel hybrid LOA-VCS metaheuristic approach with adaptive parameter tuning for network intrusion detection', Int. J. Comput. Netw. Inf. Secur., vol. 17, no. 5, pp. 31–44, Oct. 2025.

[4] D. Oluremi, A. Ali, A. Muthanna, M. Alkhalidy, R. Kumar, and M. Tiwari, 'Hybrid Metaheuristic and Deep Learning Models for Intrusion Detection in Cybersecurity Networks', 07 2025.

[5] M. S. Ahsan, S. Islam, and S. Shatabda, ‘A systematic review of metaheuristics-based and machine learning-driven intrusion detection systems in IoT’, Swarm Evol. Comput., vol. 96, no. 101984, p. 101984, July 2025.

[6] R. Parimala and S. Gunasekaran, ‘Analysis of Metaheuristic-Based Feature Selection Techniques for Intrusion Detection in Cloud Computing Environments’, Journal of Emerging Technologies and Innovative Research, vol. 13, no. 1, pp. 1–15, Jan. 2026.

[7] Scikit-learn, "RandomForestClassifier." Scikit-learn. https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestClassifier.html#sklearn.ensemble.RandomForestClassifier (accessed 31 Mar. 2026).

[8] Scikit-learn, "RandomForestClassifier." Scikit-learn. https://github.com/scikit-learn/scikit-learn/blob/fe2edb3cd/sklearn/tree/_classes.py#L707 (accessed 31 Mar. 2026).

[9] GeeksforGeeks, "XGBoost." GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/xgboost/ (accessed 31 Mar. 2026).

[10] GeeksforGeeks, "Gradient Boosting." GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/ml-gradient-boosting/ (accessed 31 Mar. 2026).

[11] GeeksforGeeks, "Gradient Descent." GeeksforGeeks. https://www.geeksforgeeks.org/machine-learning/gradient-descent-algorithm-and-its-variants/ (accessed 31 Mar. 2026).

[12] IBM, "What is an intrusion detection system (IDS)?" IBM. https://www.ibm.com/think/topics/intrusion-detection-system#437817031 (accessed 8 Mar. 2026).

[13] IBM, "What is cybersecurity?" IBM. https://www.ibm.com/think/topics/cybersecurity#7281535 (accessed 8 Mar. 2026).

[14] IBM, "What is threat detection and response?" IBM. https://www.ibm.com/think/topics/threat-detection-and-response#2086344954 (accessed 8 Mar. 2026).